Rapport de Projet : Application Android et Web Service PHP (Architecture Client-Serveur)

1. Objectif du Projet

L'objectif de ce projet était de mettre en place une architecture client-serveur complète. Il s'agissait de développer une application mobile Android capable d'envoyer et de récupérer des données depuis une base de données MySQL distante, en passant par une API (Web Service) développée en PHP.

2. Technologies et Outils Utilisés

- **Base de données :** MySQL (gérée via XAMPP et phpMyAdmin).
- **Backend (Serveur) :** PHP natif.
- **Éditeur de code (Backend) :** Visual Studio Code (VS Code) utilisé à la place de NetBeans pour une structure plus légère.
- **Frontend (Client Mobile) :** Android Studio (Langage Java et XML).
- **Bibliothèques Android :**
  - **Volley :** Pour la gestion des requêtes HTTP (réseau) asynchrones.
  - **Gson :** Pour la sérialisation et désérialisation (parsing) du format JSON vers des objets Java.

3. Étapes de Réalisation

Partie 1 : Création de la Base de Données

- Lancement du serveur local via **XAMPP** (Apache et MySQL).
- Création d'une base de données nommée school1.
- Création d'une table Etudiant comprenant les champs : id (Auto-incrémenté), nom, prenom, ville, et sexe.
- Insertion de quelques données de test directement via des requêtes SQL pour préparer la phase de test.

MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0019 seconde(s).)

```
CREATE DATABASE school1;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

Error: #1046 Aucune base n'a été sélectionnée

MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0003 seconde(s).)

```
USE school1;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

Error: #1046 Aucune base n'a été sélectionnée

MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0163 seconde(s).)

```
CREATE TABLE Etudiant ( id INT AUTO_INCREMENT PRIMARY KEY, nom VARCHAR(50), prenom VARCHAR(50), ville VARCHAR(50), sexe VARCHAR(10) );
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

2 lignes insérées.
Identifiant de la ligne insérée : 2 (traitement en 0,0031 seconde(s).)

```
INSERT INTO Etudiant (nom, prenom, ville, sexe) VALUES ('Lachgar', 'Mohamed', 'Rabat', 'homme'), ('Safi', 'Amine', 'Marrakech', 'homme');
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

1 ligne affectée.

```
UPDATE `etudiant` SET `nom` = 'Yasin' WHERE `etudiant`.`id` = 2;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

Affichage des lignes 0 - 1 (total de 2, traitement en 0,0004 seconde(s).)

```
SELECT * FROM `etudiant`
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Options supplémentaires

			id	nom	prenom	ville	sexe
Éditer	Copier	Supprimer	1	Ahmed	Belrhazi	Marrakech	homme
Éditer	Copier	Supprimer	2	Yasin	Amine	Marrakech	homme

Tout cocher *Avec la sélection :* Éditer Copier Supprimer Exporter

Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Partie 2 : Développement des Web Services PHP

Mise en place d'une architecture propre basée sur le **Pattern DAO (Data Access Object)** :

1. **Connexion :** Création de la classe Connexion.php utilisant PDO pour établir le lien avec la base MySQL locale de manière sécurisée.
2. **Modèle :** Création de la classe Etudiant.php représentant la structure d'un étudiant.
3. **Service :** Implémentation de EtudiantService.php contenant les requêtes SQL (Insertion via INSERT et Lecture via SELECT).
4. **Points d'entrée (Endpoints API) :**

- loadEtudiant.php : Service renvoyant la liste totale des étudiants au format **JSON**.
- createEtudiant.php : Service écoutant les requêtes POST, insérant un nouvel étudiant en base, et renvoyant la liste mise à jour en JSON.

- *Validation : Test des URL directement dans le navigateur pour confirmer la bonne génération du JSON.*

[{"id":1,"nom":"Ahmed","prenom":"Belrhazi","ville":"Marrakech ","sexe":"homme"},{"id":2,"nom":"Yasin","prenom":"Amine","ville":"Marrakech","sexe":"homme"}]

Partie 3 : Développement de l'Application Android

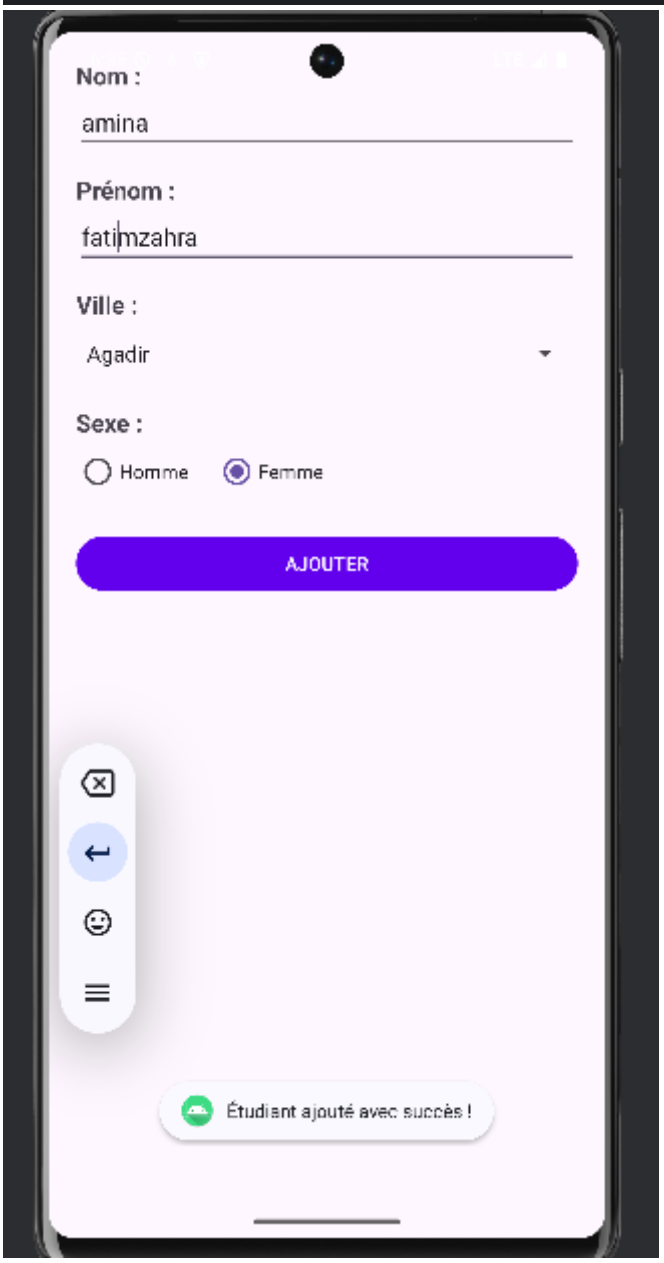
1. **Configuration :**
   - Ajout de la permission <uses-permission android:name="android.permission.INTERNET" /> et de l'autorisation du trafic local (usesCleartextTraffic="true") dans le AndroidManifest.xml.
   - Intégration des dépendances **Volley** et **Gson** dans le fichier build.gradle (Module: app).

```
35
36  dependencies {
37      implementation(libs.appcompat)
38      implementation(libs.material)
39      implementation(libs.activity)
40      implementation(libs.constraintlayout)
41      testImplementation(libs.junit)
42      androidTestImplementation(libs.ext.junit)
43      androidTestImplementation(libs.espresso.core)
44      implementation("com.android.volley:volley:1.2.1")
45      implementation("com.google.code.gson:gson:2.10.1")
46  }
```

```
gle.com/dl/android/maven2/com/android/volley/volley/1.2.1/volley-1.2.1.module, took 418 ms
aven.apache.org/maven2/com/google/code/gson/gson-parent/2.10.1/gson-parent-2.10.1.pom, took 126 ms
gle.com/dl/android/maven2/com/android/volley/volley/1.2.1/volley-1.2.1.aar, took 1 s 168 ms
aven.apache.org/maven2/com/google/code/gson/gson/2.10.1/gson-2.10.1.jar, took 1 s 284 ms
aven.apache.org/maven2/com/google/code/gson/gson/2.10.1/gson-2.10.1-javadoc.jar, took 1 s 579 ms
aven.apache.org/maven2/com/google/code/gson/gson/2.10.1/gson-2.10.1-sources.jar, took 725 ms
```

2. **Interface Graphique (UI) :**
    - Création d'un formulaire complet (activity_add_etudiant.xml) avec des champs de saisie (EditText), une liste déroulante (Spinner) pour les villes, des boutons radio (RadioGroup) pour le sexe, et un bouton de validation.
3. **Logique Métier (Java) :**
    - Création de la classe AddEtudiant.java.
    - Récupération des données saisies par l'utilisateur.
    - Utilisation de **Volley** (StringRequest en méthode POST) pour envoyer les données vers l'URL du Web Service PHP (en utilisant l'IP spéciale 10.0.2.2 de l'émulateur).
    - Réception de la réponse du serveur et utilisation de **Gson** pour transformer le texte JSON brut en une collection d'objets Java Etudiant.
    - Affichage des résultats dans la console **Logcat** de manière structurée.

```
ETUDIANT    com.example.projetws    D    Etudiant{id=1, nom='Ahmed', prenom='Belrhazi', ville='Marrakech ', sexe='homme'}
ETUDIANT    com.example.projetws    D    Etudiant{id=2, nom='Yasin', prenom='Amine', ville='Marrakech', sexe='homme'}
ETUDIANT    com.example.projetws    D    Etudiant{id=3, nom='ahmed', prenom='belrhazi', ville='Marrakech', sexe='homme'}
ETUDIANT    com.example.projetws    D    Etudiant{id=1, nom='Ahmed', prenom='Belrhazi', ville='Marrakech ', sexe='homme'}
ETUDIANT    com.example.projetws    D    Etudiant{id=2, nom='Yasin', prenom='Amine', ville='Marrakech', sexe='homme'}
ETUDIANT    com.example.projetws    D    Etudiant{id=3, nom='ahmed', prenom='belrhazi', ville='Marrakech', sexe='homme'}
ETUDIANT    com.example.projetws    D    Etudiant{id=4, nom='amina', prenom='fatimzahra', ville='Agadir', sexe='femme'}
```



4. Conclusion

Le projet est un succès. La communication entre l'application mobile (Client) et le serveur local (Backend + Base de données) est parfaitement fonctionnelle. Les données saisies sur le téléphone sont correctement insérées dans la base MySQL et la réponse JSON est correctement interprétée par l'application.